

# Greenfield MP Project Coordinator Report

---

## Repository Overview

---

**Repository:** abeer10-coder/greenfield\_mp  
**Analysis Date:** 2026-10-04  
**Project Type:** Employee Analytics and Data Warehouse System

The greenfield\_mp repository is a comprehensive employee analytics platform that combines OLTP (Online Transaction Processing) and OLAP (Online Analytical Processing) capabilities. The system tracks employee data, job history, projects, assignments, and performance reviews, with a data warehouse layer for analytical reporting.

## Architecture Findings

---

### System Components Identified:

1. **Frontend:** Streamlit-based web application ( `app/` directory)
2. **Backend:** Python/FastAPI services ( `src/` directory)
3. **Database:** - OLTP: MySQL/PostgreSQL schema ( `sql/` directory) - OLAP: Star schema data warehouse
4. **ETL:** Data transformation pipelines ( `scripts/` directory)
5. **Configuration:** Environment and deployment configs ( `.devcontainer/` , various config files)
6. **Documentation:** Comprehensive docs in `docs/` directory

### Key Architectural Decisions:

- Separation of concerns between transactional and analytical systems
- Use of star schema for data warehouse with slowly changing dimensions
- Modular design with clear boundaries between components
- Containerized development environment for consistency

## ERD Findings

---

The Entity Relationship Diagram (stored in `docs/diagrams/erd.drawio` and `docs/diagrams/erd.xml` ) reveals:

### Core Entities:

- **DEPARTMENTS:** Organizational structure

- **JOB\_ROLES**: Position definitions
- **EMPLOYEES**: Core employee information
- **EMPLOYEE\_JOB\_HISTORY**: Historical tracking of employee changes (SCD Type 2 lite)
- **PROJECTS**: Company initiatives
- **PROJECT\_ASSIGNMENTS**: Employee-project relationships
- **PERFORMANCE\_REVIEWS**: Employee evaluation data

## Relationships:

- One-to-many relationships dominate (departments → employees, employees → history, etc.)
- Many-to-many relationships resolved through junction tables (employees ↔ projects via assignments)
- Proper foreign key constraints maintaining referential integrity
- Unique keys on business-critical fields (email, department\_name, job\_title, project\_name)

## Dimensional Model Findings

---

The Dimensional Model (stored in `docs/diagrams/dimensional-model.drawio` and `docs/diagrams/dimensional-model.xml` ) implements a star schema:

### Fact Table:

- **FACT\_PERFORMANCE\_REVIEWS**: Contains metrics and foreign keys to dimensions
- Grain: One row per performance review per employee
- Measures: performance\_rating, review\_score, job\_satisfaction, etc.
- Degenerate dimension: review\_id (preserves source system ID)

### Dimension Tables:

- **DIM\_EMPLOYEE**: Employee attributes with SCD Type 2 support (start\_date, end\_date, is\_current)
- **DIM\_DEPARTMENT**: Department information
- **DIM\_PROJECT**: Project details
- **DIM\_DATE**: Time dimension for temporal analysis

### Key Features:

- Surrogate keys for all dimension tables
- Business keys preserved for traceability
- Slowly Changing Dimension Type 2 implementation for employee attributes
- Proper star schema relationships (all dimensions connect directly to fact table)

# Governance Recommendations

---

## Repository Structure Improvements:

1. Standardize directory naming conventions
2. Add clear separation between source code, configuration, and documentation
3. Implement consistent naming for configuration files
4. Add `.github/` directory for issue and PR templates

## Branching Strategy:

- **main**: Production-ready code only
- **develop**: Integration branch for features
- **feature/\***: New feature development
- **bugfix/\***: Bug fixes
- **hotfix/\***: Critical production fixes

## Branch Protection Rules:

- Require pull request reviews before merging
- Require status checks to pass
- Include administrators in protection rules
- Restrict who can push to matching branches
- Require linear history

## Pull Request Workflow:

1. Create feature branch from develop
2. Develop and test changes
3. Open PR against develop branch
4. Minimum 1 approving review required
5. Address all review comments
6. Squash and merge upon approval
7. Delete feature branch after merge

## Code Review Requirements:

- All code must be reviewed by at least one team member
- Review checklist: functionality, readability, performance, security, tests
- Automated checks must pass (linting, unit tests)
- No merging without approvals

## Merge Strategy:

- Use squash and merge for feature branches

- Use merge commit for develop to main releases
- Maintain clean, linear history on main branch
- Tag releases with semantic versioning

## Issue Templates:

- Bug Report: Clear reproduction steps, expected vs actual behavior
- Feature Request: Description, motivation, acceptance criteria
- Task: Clear definition of done, dependencies

## PR Templates:

- Summary of changes
- Related issue numbers
- Testing performed
- Screenshots/UI changes (if applicable)
- Checklist for completeness

## Testing Recommendations

---

### Integration Test Plan:

1. **UI → Backend:** Test Streamlit endpoints interact with backend APIs
2. **Backend → Database:** Verify ORM/DAO operations work correctly
3. **ETL → Warehouse:** Confirm data flows correctly from source to warehouse
4. **End-to-End Flows:** Test complete employee lifecycle from hire to review

### Performance Test Plan (Target: 100,000+ rows):

1. **Query Benchmarks:** - Simple lookups: <100ms - Complex aggregations: <2s - Reports: <5s
2. **Load Testing Approach:** - Gradual ramp-up to peak load - Sustained load testing - Stress testing to breaking point
3. **Index Recommendations:** - Primary keys on all tables - Foreign key indexes - Composite indexes for common query patterns - Covering indexes for frequent report queries
4. **Performance Metrics:** - Query response times - Throughput (queries/second) - Concurrent user capacity - Resource utilization (CPU, memory, I/O)
5. **Bottleneck Analysis Checklist:** - Database query optimization - API endpoint performance - ETL pipeline efficiency - Frontend rendering speed - Caching effectiveness

# Documentation Status

---

## Completed Documentation:

- ☒ GitHub Governance ( docs/github-governance.md )
- ☒ PR Workflow ( docs/pr-workflow.md )
- ☒ ERD Specification ( docs/erd-specification.md )
- ☒ Dimensional Model Specification ( docs/dimensional-model-specification.md )
- ☒ Data Contract ( docs/data-contract.md )
- ☒ Setup Guide ( docs/setup-guide.md )
- ☒ Architecture Overview ( docs/architecture.md )
- ☒ Data Warehouse Overview ( docs/data-warehouse.md )
- ☒ Integration Test Plan ( docs/integration-test-plan.md )
- ☒ Performance Test Plan ( docs/performance-test-plan.md )
- ☒ Repository Audit ( docs/repository-audit.md )
- ☒ Diagram Files ( docs/diagrams/ ):
  - ERD Diagram ( erd.drawio , erd.xml )
  - Dimensional Model ( dimensional-model.drawio , dimensional-model.xml )
  - Diagram README ( README.md )

## Existing Documentation:

- ☒ Original README.md
- ☒ Original ARCHITECTURE.md
- ☒ Original GIT\_WORKFLOW.md

## Risks Identified

---

1. **Data Volume Growth:** System may struggle with very large employee bases
2. **ETL Complexity:** Transformation logic may become difficult to maintain
3. **Schema Evolution:** Changing business requirements may require frequent schema updates
4. **Performance Degradation:** Analytical queries may slow as data warehouse grows
5. **Security Concerns:** Employee data requires robust access controls and encryption
6. **Operational Overhead:** Managing both OLTP and OLAP systems increases complexity

## Next Actions

---

### Immediate (0-2 weeks):

1. Implement branch protection rules in repository settings
2. Create standardized issue and PR templates
3. Set up automated testing pipeline (CI/CD)

4. Establish code review guidelines and checklist

### **Short-term (1-3 months):**

1. Implement recommended indexing strategy
2. Add comprehensive test coverage for critical paths
3. Develop data validation and quality checks in ETL pipelines
4. Implement monitoring and alerting for system performance

### **Long-term (3-6 months):**

1. Evaluate partitioning strategies for large fact tables
2. Consider implementing data archiving strategy
3. Explore advanced analytics capabilities (predictive modeling)
4. Implement role-based access control for sensitive data

## **Files Created**

---

### **Documentation Files:**

- docs/github-governance.md
- docs/pr-workflow.md
- docs/erd-specification.md
- docs/dimensional-model-specification.md
- docs/data-contract.md
- docs/setup-guide.md
- docs/architecture.md
- docs/data-warehouse.md
- docs/integration-test-plan.md
- docs/performance-test-plan.md
- docs/repository-audit.md
- docs/project-coordinator-report.md

### **Diagram Files:**

- docs/diagrams/erd.drawio
- docs/diagrams/erd.xml
- docs/diagrams/dimensional-model.drawio
- docs/diagrams/dimensional-model.xml
- docs/diagrams/README.md

## Files Modified

---

No existing files were modified - all work was completed by creating new documentation and diagram files as requested.

## Recommended Manual Review Items

---

1. **Review all generated documentation** for technical accuracy and completeness
2. **Validate diagram correctness** by opening in Draw.io and verifying relationships
3. **Check data contract** against actual database schema for consistency
4. **Review governance recommendations** with team for practicality and adoption
5. **Verify test plans** align with actual system capabilities and testing frameworks
6. **Confirm architecture diagrams** match current implementation
7. **Review data warehouse specifications** against ETL implementation
8. **Validate setup guides** by following them in a fresh environment